Field Hours

CSCE 735 Parallel Computing

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Four states of one line

You track a cache line in 4 states so two cores do not silently keep different data. Modified means this core has the only copy and it is dirty. Exclusive means the only clean copy. Shared means other cores may hold it. Invalid means do not use it. A write to a Shared line invalidates the others first. With 2 cores in Shared, one store leaves the writer Modified and the other Invalid.

You pay a coherence message for that store, not only a local write. The line is 64 bytes. Invalidating it reaches the other core even if you changed 4 bytes. Exclusive lets you write with no message when nobody else holds the line: you upgrade Exclusive to Modified locally. The state is checked on the access. If you forget that Shared requires an invalidation, you will describe a broadcast as if it had been one store.

A store to a Shared line is a coherence transaction. Exclusive can upgrade locally.

Before the store

Both cores Shared

Line is 64 bytes

Data matches

After the store

Writer is Modified

Other core is Invalid

4 bytes caused a message

Two cores, one store

1. Core 0 and core 1 both hold the line Shared. The line is 64 bytes.

2. Core 0 stores 4 bytes. It must invalidate core 1. Core 0 becomes Modified. Core 1 becomes Invalid.

3. A later read on core 1 misses and fetches the new data. If the line had been Exclusive on core 0, the store would have upgraded to Modified with no message.

Problem 1. Two cores hold a line Shared. One stores. What states do the writer and the other core end in?

Answer. The writer is Modified and the other core is Invalid. The store is not only a local write. The other copy must be invalidated first.

Problem 2. The line is Exclusive on one core and Invalid everywhere else. The core stores 4 bytes of the 64-byte line. Does it send an invalidation?

Answer. No. Exclusive means nobody else holds the line, so the upgrade to Modified is local. A Shared line would have required an invalidation even for those 4 bytes.

Lecture 2 · 90 minutes

Shared memory or a message

You pick the tool from the address spaces. Four cores on one socket share memory, so a thread publishes a result by writing a variable. Four nodes do not, so you send a message of 4096 bytes. A race is 2 threads using one address, at least one writing, with no order you enforced. A receive completes after its send. You do not hide a race with extra messages on one socket unless the copy is the point.

You pay for the choice in overhead. Spawning 4 threads to add 4 numbers spends more time on the spawn than on the adds. You put enough work in a task, thousands of operations, before you pay for another thread. A message of 8 bytes pays almost the same startup as a larger one, so you pack values into one buffer of 8 doubles instead of sending 8 messages. The tool fits when the work between synchronizations hides the synchronization.

Choose from whether an address is shared. Then make the work bigger than the handshake.

Shared socket or 4 nodes -> Threads or one message -> Pack 8 doubles -> Check for a race

Four numbers, wrong tool

1. You want the sum of 4 numbers on one socket. Spawning 4 threads costs more than the 4 adds.

2. The right cut is one thread, or a larger task with thousands of operations per thread.

3. Across 4 nodes you cannot publish the sum by writing a variable. You send a message. Eight separate 8-byte messages pay 8 startups; one buffer of 8 doubles pays 1.

Problem 1. Two threads write the same address and you enforced no order. What is that called, and does sending extra messages inside one socket fix it?

Answer. A data race. Extra messages inside one socket only help if the copy is the point. The fix on shared memory is an order you enforce, such as a lock or an atomic, around that address.

Problem 2. You need to move 8 doubles to another node. Why is one message better than 8 messages?

Answer. Each message pays startup. Eight bytes of payload barely change that startup, so 8 messages pay 8 startups. One buffer of 8 doubles pays once.

Lecture 3 · 90 minutes

A bound before you buy cores

You bound speedup before you add cores. If fraction s is serial and the rest scales with n cores, speedup is 1 divided by s plus (1 minus s) divided by n. For s = 0.2 and n = 4, the denominator is 0.2 + 0.8/4 = 0.4. Speedup is 1/0.4 = 2.5. Four cores do not give you 4. The serial 0.2 of the original time remains, and the parallel part also

shrinks to 0.2.

You compute efficiency as speedup divided by n. Here 2.5/4 = 0.625, so the cores are not full. The mechanism that sets s is whatever you left outside the parallel region: a prefix, a copy back to one core, a final reduction done serially. You measure s from a serial timing and a parallel timing. You do not guess it. Cutting s is often worth more than adding a fifth core past the 4 you already have.

Speedup is 1 over the serial fraction plus the scaled parallel fraction.

Serial fraction 0.2

Parallel 0.8 over 4 cores

Denominator 0.4

Speedup 2.5

Serial fraction 0.2, four cores

1. s is 0.2 and n is 4. The parallel fraction is 0.8, and 0.8/4 = 0.2.

2. The denominator is 0.2 + 0.2 = 0.4. Speedup is 1/0.4 = 2.5.

3. Efficiency is 2.5/4 = 0.625. A measured run below 2.5 means the parallel part did not actually shrink by 4, or s was larger than 0.2.

Problem 1. The serial fraction is 0.2 and you have 4 cores. What speedup does the formula give, and what is the efficiency?

Answer. Speedup is 1 / (0.2 + 0.8/4) = 1 / 0.4 = 2.5. Efficiency is 2.5/4 = 0.625. You do not get 4.

Problem 2. Where does the serial fraction come from if you did not guess it?

Answer. From timing. It is the work you left outside the parallel region, such as a prefix, a copy back to one core, or a serial reduction. You measure it. Adding cores does not shrink that fraction.

Lecture 4 · 90 minutes

A tree of partials

You reduce with a tree so the span is short. Eight partial sums combine in 3 steps: 4 pairs, then 2, then 1. A serial loop needs 7 additions and has span 7. The tree also uses 7 additions, but the span is 3, so enough cores finish in 3 addition times plus movement. You do not claim fewer additions. You claim a shorter chain of dependent additions.

You build a prefix when every index needs the sum of the values before it, not only the total. A serial scan of 8 elements does 7 additions and produces all 8 outputs. A parallel scan does more additions, on the order of 8 times 3 if each element climbs and descends, and the span stays a small multiple of 3. You use it to hand each worker a starting offset. A reduction alone cannot do that. The prefix can.

The tree shortens the chain. It does not reduce the number of additions below 7.

1. Eight partials

2. Pair down in 3 steps

3. Span is 3, work is 7

4. Prefix if every index needs a sum

Eight partial sums

1. You have 8 partial sums. Pairing them takes 4 additions, then 2, then 1. That is 7 additions, the same count as a serial loop.

2. The longest chain is 3 additions, not 7. With enough cores the span is 3 steps plus the movement of the partials.

3. If each of the 8 workers also needs its own starting offset, a reduction is not enough. A prefix of those 8 values does the extra work, on the order of 8 times 3 additions, and still has a short span.

Problem 1. You reduce 8 partial sums with pairing. How many additions, and what is the span?

Answer. 7 additions, the same as a serial loop, because 4 + 2 + 1 = 7. The span is 3 steps, not 7. The win is the shorter chain, not fewer additions.

Problem 2. Each of 8 workers needs the sum of everything before its own chunk. Why is a single reduction not enough?

Answer. A reduction produces one total. A prefix produces a sum at every index, which is the starting offset. The parallel prefix does more additions, on the order of 8 times 3, and keeps a short span.

Lecture 5 · 90 minutes

Blocks or a stride

You cut the data or you cut the tasks. A block gives each of 4 threads a contiguous quarter of 16 elements, so thread 0 owns indexes 0 through 3. A cyclic cut gives thread 0 indexes 0, 4, 8, and 12. If work grows with the index, the last block is heavy and 1 thread falls behind. Cyclic spreads those heavy indexes. You pick the cut from the cost of an element, not from a drawing.

You also cut for communication. A block of a grid keeps neighbors together, so a thread's edge is short. A cyclic cut makes almost every neighbor belong to someone else, and a stencil then talks on every point. For 16 points in a line and 4 threads, a block thread owns 4 consecutive points and talks at most at 2 ends. You accept imbalance only when that edge traffic is the worse problem.

Block keeps neighbors. Cyclic spreads uneven work. You cannot have both for free.

Block of 16

Thread 0 owns 0 to 3

Edge is 2 ends

Last block can be heavy

Cyclic of 16

Thread 0 owns 0, 4, 8, 12

Heavy indexes are spread

Neighbors are remote

Sixteen elements, four threads

1. Block: thread 0 gets 0, 1, 2, 3. Thread 3 gets 12, 13, 14, 15. If index 15 is the expensive one, thread 3 is the straggler.

2. Cyclic: thread 0 gets 0, 4, 8, 12. The expensive high indexes are spread over the 4 threads.

3. On a line, the block thread touches foreign data at most at 2 ends. The cyclic thread's 4 points each need neighbors it does not own.

Problem 1. Sixteen elements and 4 threads. Which indexes does thread 0 own under a block cut, and which under a cyclic cut?

Answer. Block: 0 through 3. Cyclic: 0, 4, 8, and 12. If work grows with the index, the block cut leaves the last thread heavy and the cyclic cut spreads it.

Problem 2. A stencil needs neighbors. Why can a block cut talk less than a cyclic cut on a line of 16?

Answer. A block thread owns 4 consecutive points and shares an edge at most at 2 ends. A cyclic thread does not own the neighbors of its 4 points, so almost every point needs a remote value.

Lecture 6 · 90 minutes

The line, not the address

You separate false sharing from a true race. True sharing is 2 cores writing one address, and it needs a lock or an atomic if both updates must count. False sharing is 2 cores writing different addresses on one 64-byte line. Core 0 writes byte 0. Core 1 writes byte 4. Each store invalidates the other core, though neither reads the other's counter. Coherence did what you asked, at line granularity.

You stop the bounce by separating the lines. Pad each counter to 64 bytes, or give each core a private partial and add the partials once. Four cores writing 4 adjacent counters can move the line on every store. Four cores writing 4 private partials, then adding those 4 numbers, move the line a few times instead of once per increment. The result matches only when the operation allows that combine, as a sum does. You pad or you privatize.

Different addresses can still share a line. The invalidation does not care that the program does.

True sharing

One address

Both writes must count

Lock or atomic

False sharing

Bytes 0 and 4

One 64-byte line

Pad or use 4 partials

Two counters, four bytes apart

1. Core 0 increments a counter at byte 0. Core 1 increments a different counter at byte 4. Both sit on one 64-byte line.

2. Each store invalidates the other core. The programs never read each other's counters. That is false sharing, not a race on one address.

3. Pad each counter out to 64 bytes, or keep 4 private partials and add them at the end. A sum allows that combine. A linked-list insert would not.

Problem 1. Two cores write different counters at byte 0 and byte 4 of the same 64-byte line. Is this a data race on one address?

Answer. No. The addresses differ, so it is false sharing. Each store still invalidates the other core because the line is the coherence unit. True sharing would be both cores writing the same address.

Problem 2. You have 4 cores and 4 counters that are only added. What do you change so the line does not bounce on every increment?

Answer. Give each core a private partial and add the 4 partials once, or pad each counter to 64 bytes. Both stop unrelated stores from sharing a line. Privatization is correct here because a sum can be combined.

Lecture 7 · 90 minutes

A tile that fits

You tile a matrix product so the block stays in cache. For order 8 and a tile of 4, a tile holds 16 numbers. A core loads a 4 by 4 tile of A and of B, forms a partial 4 by 4 of C, and reuses those loads inside the tile. Without a tile, the inner loop runs far enough that early values of A leave the cache before reuse. You changed the reuse, not the formula.

You count tile steps. Order 8 divided by 4 is 2, so each output tile accumulates 2 products. Tiling both dimensions gives 4 output tiles, and 4 cores can take one each. Those cores may share reads of A and B. They must not write the same tile of C. Two writers on one element are a race, not a tiling win. You assign each output tile to 1 core.

Reuse is inside the tile. The product is the same arithmetic with a better schedule.

Order 8, tile 4

16 numbers in a tile

2 steps along the inner edge

4 output tiles, 1 core each

Four output tiles

1. Order 8 divided by 4 is 2, so there are 2 tile steps along a dimension. Each tile holds 4 times 4 = 16 numbers.

2. Output tiles: 2 along rows times 2 along columns is 4. Each accumulates 2 products of input tiles.

3. Four cores can take one output tile each. They may all read A and B. If two cores write the same C tile, those elements race. Assign each of the 4 tiles to one core.

Problem 1. Matrices are order 8 and the tile is 4. How many output tiles are there, and how many input-tile products does each accumulate?

Answer. 4 output tiles, because (8/4) times (8/4) = 4. Each accumulates 8/4 = 2 products along the inner dimension. A tile holds 16 numbers.

Problem 2. Two cores write the same tile of C. Why is that not a successful parallel tiling?

Answer. Those C elements race: two writers, one address, no order. Reads of A and B may be shared. You assign each of the 4 output tiles to 1 core.

Lecture 8 · 90 minutes

A bound you can defend

You can now take a small parallel program from the machine to a bound. You can say what a store does to a Shared line on 2 cores, and pick threads or a message from whether memory is shared. You can compute speedup 2.5 for serial fraction 0.2 on 4 cores. You can replace a reduction of 8 values by a tree of span 3, pick a block or cyclic cut, and keep 2 counters off one 64-byte line.

You can also refuse a speedup the formula will not give. Four cores and a serial fraction of 0.2 cannot yield a speedup of 4. Tiling matrices of order 8 with tiles of 4 raises cache reuse and does not shrink that fraction. False sharing can push a measured speedup below 2.5 by adding coherence traffic the formula left out. You write the bound, name the traffic, and only then time the run.

Write the bound first. Traffic the formula ignored can only make the measurement worse.

1. Compute speedup 2.5

2. Remove false sharing

3. Tile for reuse

4. Then time the run

The bound and the bounce

1. Serial fraction 0.2 and 4 cores give speedup 2.5 only if the parallel part really scales and adds no extra serial traffic.

2. Two cores then write different counters on one 64-byte line inside that parallel part. Each store invalidates the other core.

3. The measured speedup can sit below 2.5. Padding each counter to 64 bytes, or using private partials, removes that traffic. Tiling an order-8 product does not change the 0.2 by itself.

Problem 1. The serial fraction is 0.2 on 4 cores, so the formula gives 2.5 if the parallel part scales. Two cores then write different counters on one 64-byte line inside that parallel part. Does the formula still promise 2.5, and what does a store do?

Answer. No. The 2.5 assumes the parallel fraction scales and does not add coherence traffic. Each store to a shared line invalidates the other core, so the line bounces. Measured speedup can fall below 2.5. Pad to 64 bytes or use private partials.

Problem 2. You tile order-8 matrices with tiles of 4 and you also owe a prefix so 8 workers can find their starting offsets. How many output tiles are there, and why does a reduction not hand out those offsets?

Answer. There are 4 output tiles, from (8/4) times (8/4). A reduction yields one total, not a sum at each index. A prefix of the 8 chunk sizes produces the offsets. Tiling and the prefix solve different problems, and neither erases a serial fraction of 0.2.